University of Tübingen

Faculty of Science
Wilhelm-Schickard Institute for Computer Science (WSI)

Master's Thesis in Computer Science

# Hardware-Aware Neural Architecture Search for Capsule Endoscopy

Divya Schäfer

21st June 2026

**Reviewers**

Prof. Dr. Oliver Bringmann
(Head of the Embedded Systems Chair)
Wilhelm-Schickard Institute for Computer Science
University of Tübingen

Prof. Dr.-Ing. Thomas Küstner
(Head of the MIDAS Laboratory)
Department of Radiology
University Hospital of Tübingen

**Supervisor**

Moritz Reiber
Wilhelm-Schickard Institute for Computer Science
University of Tübingen

# Contents

# 1 Introduction

# 2 Background

## 2.1 Related Work

## 2.2 The Target Hardware

The target platform deploys an UltraTrail low-power neural network accelerator developed for edge devices by Paul Palomero Bernardo and his colleagues at the University of Tübingen, Germany. [1], [2]. With the aim of streamlining computation across a range of temporal convolutional and residual networks, the design of the accelerator is based on a fixed workflow made up of a convolutional or linear layer, optionally followed by additional operations consisting of batch normalisation, a ReLU activation function and global average pooling, in that order [1: 6].

The computations pertaining to the convolution or linear operators are performed by a MAC (Multiply-Accumulate) array of dimension $DIM^2$, such that:

$$DIM \in \{2^k \mid k \in \mathbb{N}_{0 \leqslant k \leqslant 4}\} \tag{2.1}$$

holds true. Thus, the set of valid values for $DIM$ is $\{1, 2, 4, 8, 16\}$.

The feature maps are unrolled and each in-channel drawn from a feature memory unit (FMEM) is multiplied by its corresponding weight accessed from the weight memory (WMEM) and added to its respective intermediate sum, which is then updated and stored in the LMEM (local memory unit). The value for each unrolled `out-channel` is thus produced by summing up the product of each `in-channel` with the weight stored for the corresponding `(in-channel, out-channel)` pair stored in the two-dimensional MAC array. The post-convolutional operations (batch-normalisation, ReLU and average pooling) are executed by an Output Processing Unit (OPU) [1: 6-8], Palomero Bernardo (personal communication, 2025).

A crucial mechanism employed by the UltraTrail accelerator to ensure efficient computation is quantisation, which we turn our attention to in the next section.

## 2.3 Quantisation

### 2.3.1 Motivation

Edge devices, such as the endoscopy capsule, need to make complex computations in time-constrained circumstances and with access to limited memory capacity. Quantisation is a mechanism through which computationally expensive floating point inputs are converted into integers, so as to enable efficient computation with minimal loss of accuracy.

Of the available quantisation techniques, two are most prominent and commonly used: Post-Training Quantisation (PTQ) and Quantisation-Aware Training (QAT) [4: 431], [5]. PTQ is applied

at inference time to convert (typically 32-bit) floating point values to integers (usually of 8-bits) [6]. However, applying quantisation exclusively at inference time can lead to a significant loss of accuracy, if the weights computed during training have only seen continuous floating point values. QAT addresses this inadequacy by applying a *fake quantiser* on the inputs of such operations that will be quantised to integers using PTQ. We will focus on QAT in this thesis, since it is this quantisation technique that we implemented and that is relevant to NAS. In the following section we shall see what a fake quantiser is, how QAT is applied and why it helps prevent accuracy degradation caused by quantisation during inference.

### 2.3.2 Quantisation-Aware Training

The goal of QAT is to train a model and update its weights under circumstances that mimic quantisation at inference time, while minimising the effect of quantisation on prediction accuracy. The central piece of QAT is a fake quantisation module, which consists of two major operations:

(i) `quantise`, and (ii) `dequantise`.

These two operations are applied in the forward pass, while in the backward pass, *Straight Through Estimation* (STE) is applied, which means that the input is allowed to pass through without any effect on the gradient of the function. In this section, we will look at QAT at a general level, while in the next section we shall turn our attention to the specific UltraTrail-compatible version of QAT.

#### The Forward Pass

The first operation that is applied during the forward pass is `quantise`, and it depends on four variables: the original, unquantised input $x \in \mathbb{R}$, the scaling factor $s \in \mathbb{R}$, the zero-point $z \in \mathbb{R}$, as well as the target number of bits, $n \in \mathbb{N}$. The input is centred by subtracting the zero-point from it and then scaled by $s$. The resulting value is rounded.

$$r = \left\lfloor \frac{(x - z)}{s} \right\rceil \tag{2.2}$$

In order to prevent outliers from distorting the mapping of quantised values, the new integer values are clipped to within a range permitted by $n$ bits, i.e. between $-2^{n-1}$ and $2^{n-1} - 1$.

$$q = \min(\max(r, -2^{n-1}), 2^{n-1} - 1) \tag{2.3}$$

After quantisation, the resulting integer is converted back to a float value via the `dequantise` operation.

$$\hat{x} = q \cdot s + z \tag{2.4}$$

Essentially, `dequantise` reverses the scaling by multiplying $q$ by $s$ and then adding back the zero-point, $z$[4: 433]. But it does not reverse the rounding and clipping operations. Thus, in general, $\hat{x} \neq x$ holds. This incomplete reversal of the `quantise` operation applied to quantised inputs results in a new set of float values that are discrete and clipped within a limited range determined by the number of bits, $n$. Since the model will have to work with discrete values (integers) at

inference time, the aim of QAT is to discretise inputs during training so that model weights are optimised to make predictions based on such inputs, while minimising any degradation of accuracy by still using floats.

A further piece of complexity is added in the case of networks that use batch-normalisation. At inference time, normalisation is 'folded' into the computation of the convolution or linear layer, which incudes quantisation. This should be simulated during training by scaling the layer's weights and biases by the batch mean and variance. [5: 2709].

#### The Backward Pass

We noted in the previous subsection (2.3.2) that during training, QAT applies the scaling, rounding and clipping the inputs. However, since the rounding and clipping operations are not differentiable, the identity function is applied to the inputs instead of differentiating them. Let $\mathcal{I}$ represent the identity function and $x$ be any input. Then, it follows that:

$$\frac{\partial \mathcal{I}(x)}{\partial x} = 1 \tag{2.5}$$

The upstream gradients are multiplied by 1 and hence are passed to the next node of the computation graph unchanged.

### 2.3.3 UltraTrail QAT

The UltraTrail requires a specific, streamlined architecture based on `Conv-BN-Relu` blocks for compatibility. Based on this requirement, Palomero Bernardo et al. specify the precise workflow that fake-quantisation in the forward pass needs to follow:

1. First, the weights fed to the convolution or linear operator must be quantised.
2. The bias is quantised and added to the output of the convolution.
3. Finally, the output of the activation function that is applied at the end of the `Conv-BN-Relu` layer is quantised.

For the UltraTrail quantiser, the zero-point $z$ is taken to be 0, and can thus be left out of the computation. The scaling variable $s$ is specified to be $1/2^{n-1}$. Taken together, the quantiser operator proposed by Palomero Bernardo et al. can be expressed as follows [1: 3]:

$$r = \left\lfloor \frac{x}{1/2^{n-1}} \right\rceil = \lfloor x \cdot 2^{n-1} \rceil$$

$$q = \min(\max(r, -2^{n-1}), 2^{n-1} - 1) \tag{2.6}$$

Finally, the dequantise operation is applied to $q$:

$$\hat{x} = \frac{q}{2^{n-1}} \tag{2.7}$$

In the backward pass, STE is employed to ensure a smooth flow of gradients.

## 2.4 The Previously existing Software Framework

### 2.4.1 HANNAH

HANNAH (Hardware Accelerator and Neural Network searcH) is a Python-based coding framework for hardware accelerator design and hardware-aware neural networks developed at the Chair of Embedded Systems at the University of Tübingen, Germany. HANNAH contains a sub-framework for supporting Neural Architecture Searches. In this section, a few of the most salient aspects of the NAS sub-framework as well as other relevant portions of HANNAH will be briefly described. The tools and code discussed in this section have been programmed previously by members of the Chair of Embedded Systems, and not by the author of this thesis.

### 2.4.2 The NAS Sub-Framework

#### Defining Search Spaces

A Neural Architecture Search begins with creating a **search space**. A search space is a Directed Acyclic Graph consisting of nodes that represent input or weight tensors as well as operations conducted on these tensors such as 2D-Convolution, ReLU or quantization. The directed edges between these nodes represent the flow of data. [7]. Functionally, the construction of search spaces enables us to define the range of possible values the hyperparameters of the target neural networks may assume. Specifically, these hyperparameters include the number of layers in the network, possible kernel sizes, stride lengths and channel dimensions.

HANNAH's NAS framework supports the definition of valid hyperparameter values for search spaces via various kinds of parameter expressions. Two of the most important of these are `IntScalarParameter` and `CatagoricalParameter`. The former is used to define ranges of integer values and has the following signature:

```
IntScalarParameter(
min: Union[int, IntScalarParameter],
max: Union[int, IntScalarParameter],
step_size: int = 1,
name: Optional[str] = "",
rng: Optional[Union[np.random.Generator, int]] = None
)
```

Typically, IntScalarParameters are used to define hyperparameters that have a large number of possible integer values within a given range, such as channel shapes. An example usage would be as follows:

```
out_channels = IntScalarParameter(min=32, max=256, step_size=16,
name='out_channels')
```

The parameter expression `CatagoricalParameter`, on the other hand, is used to define a discrete number of valid choices for a given parameter. Its signature is as follows:

```
CategoricalParameter(
choices: Sequence[Any],
name: Optional[str] = "",
```

```
    rng: Optional[Union[np.random.Generator, int]] = None
    )
```

Typical use cases for `CatagoricalParameter` include the definition of valid values for `kernel_size` and `stride` or `groups`. For instance, values allowed for `kernel_size` may be defined as follows:

```
kernel_size = CategoricalParameter(name='kernel_size', choices=[1, 3, 5])
```

The parameterisation tools described thus far enable the definition of node-level hyperparameters such as kernel size, stride and number of out-channels. However, it is often desirable to render graph-level structures as searchable parameters as well. For instance, we may want our search space to include networks with varying numbers of layers or we may wish to alternate the choice of operations at certain layers. In HANNAH, creating branches of the search graph itself can be implemented using the `ChoiceOp`. This is a node that defines a choice of two or operations, exactly one of which will be chosen per network at execution time. This allows for the creation of alternate paths in the search space graph [7]. An example use case is varying the number of layers in the search space. An example realisation is as follows:

```
# implementation concept derived from
# hannah.models.embedded_vision.models and
# hannah.models.resnet.models.resnet

exits = []

for i in range IntScalarParameter(
min=3,
max=8,
name='num_internal_blocks').max:
int_layer = capsule_net.conv_relu(layer_input=layer,
out_channels=out_channels.new(),
kernel_size=kernel_size.new(),
stride=stride.new(),
groups=groups.new())

exits.append(int_layer)
layer = int_layer

out = ChoiceOp(*exits, switch=num_internal_blocks)()
```

### Constraining Search Spaces

Even carefully designed search spaces may allow for the creation of networks with parameters that could lead to incorrect or inefficient behaviour, or that may be incompatible with the target hardware platform. Weeding out such networks after search completion is cumbersome, computationally wasteful and error-prone. Instead, we need a mechanism to build constraints into our search space, so that every network explored conforms to our requirements.

HANNAH's NAS framework provides a random-walk constraint solver that enables us to do exactly this. The `init` method, which defines the construction of an object belonging to the `RandomWalkConstraintSolver` class, is as follows:

```
class RandomWalkConstraintSolver:
def __init__(self, max_iterations=5000) -> None:
self.max_iterations = max_iterations
self.constraints = None
self.solution = None
```

The random-walk constraint solver is a greedy solver that uses random jumps to improve its search space exploration. It thus employs a simple solving mechanism that may not be suitable for very complex constraint spaces, but is usually sufficient for our purposes.

Constraints can be defined using the `cond()` method for the hyperparameters of every operator decorated with `@parametrize`, such as `Conv1d`, `Conv2d` and `Linear`. For instance, the following code snippet shows how a search space can be constrained so as to ensure that only valid kernel sizes are chosen at every layer of all networks sampled during a NAS.

```
# instantiate parameterised operator
conv = Conv2d(stride=stride, dilation=dilation, groups=groups)

# define constraint on the operator using cond() method
conv.cond(kernel_size <= input_shape)
```

All the constraints pertaining to the same operator are collected into a list and passed on to the `RandomWalkConstraintSolver`, which then attempts to find a solution that satisfies all these constraints.

### 2.4.3 Hidden Markov Models and The Viterbi Algorithm

Recent work by Werner et al. [8] has proposed using Hidden Markov Models (HMM) in combination with the Viterbi algorithm to improve CNN prediction of organs labels based on capsule endoscopy images. The classification of capsule endoscopy images should ordinarily obey the order in which the endoscopy capsule moves through the GI tract. Thus, images of the oesophagus should ordinarily appear first, followed by images of the stomach, then of the small intestine (small bowel) and finally, of the large intestine (colon). But CNNs make predictions based on each image they see, and they have no concept of order or temporality. This inability of CNNs to recognise temporal order is a frequent cause of noise in capsule endoscopy image prediction. This is where HMMs can help.

#### What is an HMM?

An HMM is a finite state machine that can be defined as follows:

**Definition 2.1.** *An HMM is a 4-tuple*

$$M = \{H, A, B, \pi\}$$

*with:*
*H: a finite, non-empty set of hidden states*
*A: a matrix such that element* $a_{ij} \in A$ *is the transition probability from* $h_i \in H$ *to* $h_j \in H$
*B: a sequence of emission probabilities such that* $b_i(o_t)$ *is the probability of the hidden state being*

*$h_i \in H$ given observation $o_t$*
*$\pi$: A sequence of initial probabilities such that $\pi_i$ is the probability of $h_i \in H$ being the initial state*

In simpler terms an HMM is a probabilistic finite-state machine that represents the relationship between unknown, independent hidden states and known, dependent observations. Given an HMM and a sequence of dependent observations, we should be able to compute the most likely sequence of hidden states.

### The Viterbi Algorithm

Computing the most likely sequence of hidden states naïvely is computationally expensive. But the Viterbi algorithm makes this computation much more efficient. Furthermore, the algorithm can be made computationally more stable and economical if we work in logarithmic space [9]. We thus replace multiplying probabilities with adding their logarithms instead. The Viterbi algorithm may be divided into three phases: 1. initialisation 2. forward computation, and 3. backward computation (retracing). The algorithm is presented below in pseudocode (adapted from [9]).

**Algorithm 2.1:** Logarithmic Viterbi Decoder

**Input:**
$Y$: observations,
logP: priors
logA: transition matrix
logB: emission matrix

**Output:**
Most likely sequence of hidden states
$N \leftarrow |\text{logP}|$
$M \leftarrow |Y|$
logT1 $\leftarrow N \times M$matrix `// Store probabilities of likely path`
bestStates $\leftarrow N \times M$ matrix
result $\leftarrow$ sequence of length $M$
`// Initialisation`
**for** $1 \leq i \leq N$ **do**
  logT1$[i, 1] \leftarrow$ logP$[i]$ + logB$[i, Y[1]]$

`// Forward Phase (Recursion)`
**for** $2 \leq i \leq M$ **do**
  **for** $1 \leq j \leq N$ **do**
    bestStates$[j, i] \leftarrow \arg\max_{1 \leq k \leq N}$ ( logT1$[k, i-1]$ + logA$[k, j]$ )
    logT1$[j, i] \leftarrow \max_{1 \leq k \leq N}$ ( logT1$[k, i-1]$ + logA$[k, j]$ ) + logB$[j, Y[i]]$

`// Backward phase (retracing)`
**for** $M \geq i \geq 2$ **do**
  **if** $i = M$ **then**
    result[M] $\leftarrow \arg\max_{1 \leq k \leq N}$ logT1$[k, M]$
  **else**
    currentState $\leftarrow$ result$[i]$
    result$[i-1] \leftarrow$ bestStates[currentState, $i$]
**return** *result*

In the final phase, use the transition matrix computed in the forward phase to compute the sequence of states that best accounts for our observation sequence.

Assume the following mapping of organs to integer labels:

$$\begin{aligned} \text{oesophagus} &\leftrightarrow 0 \\ \text{stomach} &\leftrightarrow 1 \\ \text{small intestine} &\leftrightarrow 2 \\ \text{large intestine} &\leftrightarrow 3 \end{aligned}$$

Based on this mapping, assume the following series of CNN organ predictions:

$$0, 1, 1, 0, 1, 2, 1, 1, 1, 1, 2, 1, 1, 2, 2, 2 \ldots$$

## 2.5 Conclusion

# 3 Methods and Tools

## 3.1 The UltraTrailNet class

Our purpose in programming the `UltraTrailNet` class is to provide pre-fabricated blocks of code that can be customised within a constrained space to instantiate layers of a NAS search space. We used the constraint solving system described in the previous chapter (Chapter 2) to implement constraints that ensure adherence to the requirements of the target hardware platform, as well as to proscribe incorrect network behaviour.

The constructor ( `init` method) of the `UltraTrailNet` class is as follows:

```
DIMS = {1,2,4,8,16} # MAC-array dimension = 2^k, k <= 4

class UltraTrailNet:
def __init__(self, initial_input, hardware_dim=8):
self.initial_input = initial_input
self.hardware_dim = hardware_dim
assert hardware_dim in DIMS, "hardware_dim must be in DIMS = {1,2,4,8,16}"
```

As can be seen, an `UltraTrailNet` instance can be defined with an initial input ( `initial_input` ) and a hardware dimension ( `hardware_dim` ). The former enables us to enforce conditions specific to the input layer. For example:

```
is_original_input = layer_input is self.initial_input
if is_original_input:
groups = 1
```

The other argument `hardware_dim` represents a single row or column dimension of a square MAC (Multiply-and-Accumulate) array. This is necessary for compatibility with the target hardware platform. The requirements of the target hardware and the implementation of constraints required for compliance with it will be discussed in greater detail in the next chapter.

The `UltraTrailNet` class also contains the following methods: `adaptive_avg_pooling` , `linear` , `conv_relu` , `depthwise_conv_relu` and `depthwise_separable_conv_relu` .

`_quantize` : This method is intended for internal use and calls the `ULtraTrailSTEQuantize()` class, which inherits from the `Requantize` class of `hannah.nas.functional_operators.py` and implements a fake quantiser.

```
# import at the top of ultra_trail.py:
from hannah.nas.ultra_trail.qat import UltraTrailSTEQuantize

# method within UltraTrailNet class:
def _quantize(self, input):
return UltraTrailSTEQuantize()(input)
```

`_conv` : Also intended for internal use, this method forms the central node through which all convolution operations must pass, since it implements the constraints necessary for correct network design as well as compliance with the target AI accelerator (see Section 3.2). It also calls either the `Conv1d` or the `Conv2d` classes from `hannah.nas.functional_operators` based on the value passed for the argument `conv_dim` . If `_conv` is called with `conv_dim=1` , `Conv1d` is called, else `Conv2d` is called. The default value of `conv_dim` is 2. The `_conv` method also takes in a number of other arguments that are used to parameterise the convolution, such as `kernel_size` , `stride` and `groups` . The full signature of the `_conv` method is as follows:

```
    def _conv(self,
    layer_input,
    out_channels,
    kernel_size,
    stride,
    groups=1,
    dilation=1,
    conv_dim=2,
    depthwise=False):
```

`batch_norm` : As a network trains over multiple layers, its input distribution tends to adapt more strongly to the training dataset and it vagaries, even while the conditional distribution of the input with respect to the true output remains unchanged. By adjusting the input values of each batch to its respective mean and variance, batch normalisation prevents overfitting of the model to the training dataset and helps aid faster convergence towards the target function [10]. Currently, `UltraTrailNet` is equipped with a basic implementation of batch normalisation that calls the `BatchNorm` class from `hannah.nas.functional_operators` :

```
    @scope
    def batch_norm(self, layer_input):
    # https://stackoverflow.com/questions/4488744/
    # pytorch-nn-functional-batch-norm-for-2d-input
    n_channels = layer_input.shape()[1]
    running_mu = Tensor(name='running_mean', shape=(n_channels,),
    axis=('C',))
    running_std = Tensor(name='running_std', shape=(n_channels,),
    axis=('C',))
    return BatchNorm()(layer_input, running_mu, running_std)
```

This implementation is sufficient for the purposes of approximating the effects of batch normalisation during NAS. However, it does not adhere to the constraints imposed by the target AI architecture [1: 3-4]. Implementing a hardware-compliant version would be complex and require introducing fairly significant modifications to HANNAH, yet would make minimal difference to the overall efficacy of NAS. We therefore arrived at the conclusion that it would be more efficient to forgo the implementation of a hardware-aware batch normalisation method within `UltraTrailNet` for the purposes of this thesis.

`relu` : This method calls the `Relu` class from `hannah.nas.functional_operators` and applies the ReLU activation to the input. Mathematically, ReLU maps any input less than

zero to zero, while in all other cases, the input is mapped to itself.

$$\mathrm{ReLU}(x) = \begin{cases} 0, & \text{if } x < 0 \\ x, & \text{otherwise} \end{cases} \tag{3.1}$$

The purpose of activation functions, in general, is to help a model learn non-linear functions. We use the `ReLU` activation in `UltraTrailNet` for the hidden (internal) layers because of its compatibility with the target AI accelerator.

`adaptive_avg_pooling` : This method uses `AdaptiveAvgPooling` from `functional_operators.py` , which serves as a flattening layer. It does so by calling `torch.nn.functional` 's methods for adaptive average pooling with an `output_size` of $(1, 1)$. This is read by `torch.nn.functional` as an output size of 1 in the case of `adaptive_avg_pooling1d` ([11]) or as $(1, 1)$ in the case of `adaptive_avg_pooling2d` [12]. The output produced by `adaptive_avg_pooling` is an array composed of one output unit (or pixel) per input feature map (or `in-channel`).

`linear` : This method applies the `Linear` class from `torch.nn.functional` to implement a linear operator, which is a special kind of convolution with `kernel_size=1` and `stride=1` . In image classification CNNs, the last layer is typically a fully-connected or linear layer, and although the method `linear` can be inserted anywhere in a network, it has been programmed specifically with this use-case in mind. The arguments of `linear` include a boolean flag `is_last` , with a default value of `True` , and a warning is issued in case this flag is set to `False` .

```
    if not is_last:
    warnings.warn("WARNING: Either a linear layer is not the\
    last layer or the is_last flag is incorrectly set.")
```

As we shall see in the following section, knowing the position of a layer is essential for the correct application of constraints, both in the case of regular convolutional layers as well as for linear layers.

`conv_relu` : This is a block construction method, whose signature is as follows:

```
    @scope
    def conv_relu(
    self,
    layer_input,
    out_channels,
    kernel_size,
    stride,
    groups,
    dilation=1,
    conv_dim=2,
    bn=False,
    depthwise=False):
```

As a block constructor, `conv_relu` calls other `UltraTrailNet` methods that are responsible for implementing atomic operations. First, it calls the `_conv` method, passing on all its own arguments to `_conv` , except for the boolean flag `bn` . Calling `_conv` ensures that all necessary parameter constraints are imposed on the layer. If `bn=True` , it calls the `batch_norm` method to insert a batch normalisation operator immediately following the

convolution operation, and then finally, it calls `relu` to apply the ReLU activation on the output of `batch_norm`, in case `bn=True` or otherwise to the output of `_conv` itself.

`depthwise_conv_relu`: This method constructs a depthwise convolution layer, which is a convolution layer in which the dimensions of both the `in-channels` as well as the `out-channels` must equal the number of groups. The `depthwise_conv_relu` method calls `_conv` with `depthwise=True`, and the `_conv` method enforces the definitional constraint governing depthwise layers as follows:

**Constraint 3.1: Depthwise Layers**

$$|\text{channels}_{in}| = |\text{channels}_{out}| = |\text{groups}|$$

This constraint is enforced within the dedicated method `depthwise_conv_relu` as follows:

```
    in_channels = layer_input.shape()[1]

    groups = in_channels
    out_channels = in_channels
```

After fixing the values of `in_channels`, `out_channels` and groups, `depthwise_conv_relu` calls the `_conv`, so that all other constraints that must apply to all convolution layers is applied to the depthwise layer as well.

`depthwise_separable_conv_relu`: Depthwise-separable layers are compound layers composed of one depthwise convolution layer followed by a piecewise convolution layer, which is a $1 \times 1$ convolutional layer with `kernel_size=1`, `stride=1` and `groups=1`. Taken together, depthwise-separable convolution combines the benefits of a layer in which each output pixel is influenced by one whole input feature map with those of an additional layer in which each output feature is influenced by exactly one corresponding input pixel from each feature map. At the same time, it is computationally cheaper than most other grouped convolutions. In `UltraTrailNet`, the method `depthwise_separable_conv_relu` calls `depthwise_conv_relu` for the first, depthwise layer and following that, it calls `conv_relu` for the second, pointwise layer with the arguments `kernel_size=1`, `stride=1` and `groups=1`.

As is clear from the above description of the various methods of the `UltraTrailNet` class, prefabricated blocks like `conv_relu` and `depthwise_conv_relu` call the `conv` method, which applies the constraints necessary for compatibility with the target AI accelerator. The `linear` method, on the other hand, applies constraints on its own. In the next section, we shall examine the constraints necessary for compliance with the UltraTrail AI accelerator as well as their implementation in `UltraTrailNet` in greater detail.

## 3.2 UltraTrail Constraints

Designing software for deployment on specific target platforms necessitates ensuring adherence to the constraints imposed by the target hardware. In the present context, we wish to design neural architectures via NAS that are guaranteed to conform to the constraints and specifications of the target platform. Rather than running searches over spaces that include all sorts of architectures—many of which may not conform to the specifications of the target hardware—our goal

should be to ensure that searches are conducted within a constrained design space of architectures that conform to the constraints imposed by the target hardware platform.

### 3.2.1 Fixed CONV-BatchNorm-ReLU Blocks

The need for fixed blocks consisting of a convolution operator followed by batch normalisation and finally applying a ReLU activation (CONV-BatchNorm-ReLU) follows directly from the fixed workflow design of the UltraTrail target hardware.

We can summarise this constraint as follows:

**Constraint 3.2: Fixed Layers**

Each convolutional layer must consist of fixed CONV-BatchNorm-ReLU blocks.

In section 3.1, we have seen that the `UltraTrailNet` class contains a `conv_relu` method that implements such a block.

### 3.2.2 Channel Dimensions

The number of `in-channels` to and `out-channels` from every layer must each be multiples of $DIM$, where $DIM$ is the number of MAC-units in a single row or column of the UltraTrail AI Accelerator's lone MAC-array. This constraint follows from the fact that the MAC-Array iteratively proceses $DIM$ number of unrolled `in-channels` to compute the output of $DIM$ unrolled `out-channels` at a time.

This constraint is implemented within the `_conv` method `UltraTrailNet` class.

Thus, for each layer's outgoing channels, we have:

**Constraint 3.3: Out-Channel Dimensions**

The number of `out-channels` from each layer must be a multiple of $DIM$.

The above constraint is implemented as follows:

```
conv.cond(out_channels % self.hardware_dim == 0)
```

And similarly for `in_channels`:

**Constraint 3.4: In-Channel Dimensions**

The number of `in-channels` to each layer must be a multiple of $DIM$.

Once we ensure that the number of `out-channels` from layer $l$ is a multiple of $DIM$, the number of `in-channels` going into layer $l+1$ is automatically constrained, since the out-channels of $l$ are usually fed as `in-channels` to $l+1$ (unless $l$ is the final layer of a network or sub-network). We nevertheless explicitly enforce the dimensionality constraint on `in-channels` as well:

```
in_channels = layer_input.shape()[1]

if not is_original_input:
conv.cond(in_channels % self.hardware_dim == 0)
```

There are two justifications for this explicit enforcement. The first is that it would enable a seamless extension of the UltraTrailNet logic to include residual layers with skip connections. Secondly, it acts as an additional sanity check at negligible computational cost, given that `assert` statements do not function as desired within such symbolic search spaces that that have yet been instantiated with concrete values.

Note that the original input to the network, identified with the boolean `is_orginal_input`, is excluded from the enforcement of this constraint, since the number of channels in the input layer in the case of image inputs is typically 3.

### 3.2.3 Handling of Padding

The UltraTrail AI accelerator supports binary mode selection for padding: either padding is not enabled (`padding = 0`) or padding is enabled (`padding = 1`). In the latter case, as long as the kernel fits completely within an input channel, all the units of the MAC arrays are used for computation. On the other hand, on time-steps where certain input pixels are not covered by the kernel, the number of MAC units in use is reduced accordingly. This implicitly implements padding with zeroes, while saving on energy use. The following operational constraint for our software platform thus applies:

**Constraint 3.5: Padding**

Padding $\in \{0, 1\}$ is permitted, such that `padding=0` means that no padding is applied and `padding=1` means that zero-padding is applied.

### 3.2.4 Handling of Depthwise Layers

We have discussed depthwise and depthwise-separable convolution layers and the methods devoted to their implementation in the previous chapter (see Chapter 2, Section **??**). In this section, we look at how these layers are realised such that hardware-compatibility is maintained.

The target UltraTrail accelerator realises depthwise layers by iterating over the diagonal of the MAC-array until all `in-channels` and `out-channels` have been processed. The feasibility and operational correctness of this approach follows from the square-shaped arrangement of MAC-units in the MAC-Array, as well as from the fact that the previously discussed constraints regarding channel dimensions being multiples of $DIM$ (see Constraint 3.3, Constraint 3.4) holds in the case of depthwise layers as well. Furthermore, because of the hardware-imposed dimensionality constraints (3.3, 3.4) and the definitional constraint of depthwise layers (see Constraint 3.1), it follows that for depthwise layers, the number of groups must be a multiple of $DIM$ as well. Within the `_conv` method, this constraint is enforced for depthwise layers as follows:

```
if depthwise:
conv.cond(groups % self.hardware_dim == 0)
```

Here we encounter another instance of an additional or redundant constraint acting as a sanity check. We already ensure that `in_channels = out_channels` are multiples of $DIM$, so setting `groups` as equal to `in_channels` should automatically ensure that `|groups|` is a multiple of $DIM$. But the constraint enforced via `conv.cond()` acts as a computationally cheap `assert` statement.

### 3.2.5 Implementing UltraTrail QAT

Another significant aspect of realising a hardware-aware NAS framework is the implementation of QAT, which as we noted in Chapter 2 is essential to effectively train networks that will use quantised inputs at inference time. For this, we implemented a *custom operator* to act as our quantiser.

#### What is a custom operator?

If we want Pytorch to treat a compound operation as a single node in the computation graph, we need to define it as a custom operator. A compound operator is composed of multiple unitary `torch` operations, which Pytorch would normally trace through while executing `torch.compile`, treating each component operator as a separate node. But we want our quantiser to be recognised as a single, opaque operator and be represented as such in the training logs. Creating a custom quantiser operator fulfils this requirement [13].

#### Creating a custom quantise operator

Pytorch offers a package called `torch.library` that contains tools for creating and testing custom operators [14]. We need to implement a function that computes the QAT forward pass as described in 2.3.3, and wrap this function into a custom operator so that Pytorch treats it as a unitary operator and does not trace inside it. `torch.library` offers a wrapper method `torch.library.custom_op` that serves this purpose.

```
import torch.library
import torch

@torch.library.custom_op(name="ekut::quantize", mutates_args=())
def quantize(x: torch.Tensor, num_bits: int) -> torch.Tensor:
scale = 2 ** (num_bits - 1)
scaled_val = torch.round(x * scale)
q_min = -2 ** (num_bits - 1)
q_max = (2 ** (num_bits - 1)) - 1
clamped_val = torch.clamp(scaled_val, q_min, q_max)

return clamped_val / scale
```

The argument `name` passed to the wrapper consists of a namespace (`ekut`) followed by the function name (`quantize`). The `name` serves as an identifier for pytorch packages such as `torch.fx` and `torch.export`. The second argument `mutates_args` is is a list of arguments that the function directly mutates. In our case, this is an empty list, since we do not reassign a value to the input `x`, and `num_bits` is used to compute the scale, but is not mutated itself [13].

Next, we need to register a *fake tensor* implementation for our new custom operator. This defines the effect of the operator on an empty, abstract tensor with regard to attributes such as `shape` and `dtype` so that `torch.export` can correctly trace through the operator , [15].

```
@quantize.register_fake
def _(x: torch.Tensor, num_bits: int) -> torch.Tensor:
return torch.zeros_like(x)
```

Finally, we need a function that defines the custom operator's behaviour in the backward pass, and register this function using `torch.library.register_autograd`. In order to realise STE, the `backward` function simply returns the upstream gradient `grad` unchanged for the input `x` that is passed to the `quantize` function. `None` is returned as the gradient of `quantize`'s second argument, `num_bits`.

```
def backward(ctx, grad):
return grad, None

torch.library.register_autograd("ekut::quantize", backward, setup_context=None)
```

In implementing STE, the backward pass is in no way dependent on what is computed in the forward pass. Therefore, `setup_context` is `None` and the `ctx` argument of the backward function remains unused.

### Wiring up UltraTrail QAT

In `hannah.nas.ultra_trail.qat`, we create a new class `UltraTrailQuantiser` that inherits from `nn.Module` and returns the custom quantiser in its forward method.

```
class UltraTrailQuantizer(nn.Module):
def __init__(self, num_bits=8):
super().__init__()
self.num_bits = num_bits

def forward(self, input):
return torch.ops.ekut.quantize(input, self.num_bits)
```

Additionally, we create a class `UltraTrailSTEQuantize` that inherits from `Requantize` in `hannah.nas.functional_operators`, which in turn inherits from `Op`. `UltraTrailSTEQuantize` is initialised with `UltraTrailQuantizer()` and `Requantize` calls `self.quantize` in its forward method:

```
class UltraTrailSTEQuantize(Requantize):
def __init__(self):
super().__init__()
self.quantize = UltraTrailQuantizer()
```

```
def _forward_implementation(self, *operands):
  return self.quantize(operands[0])
```

Finally, `UltraTrailSTEQuantize` is called by `UltraTrailNet`'s `_quantize` method (see 3.1).

### 3.2.6 Significance of UltraTrailNet

By using HANNAH's in-built `RandomWalkConstraintSolver`, we are able to apply constraints so that all search space layers constructed as members of the `UltraTrailNet` class are automatically guaranteed to adhere to these constraints. This, in turn, ensures that all models produced by a NAS, whose networks are instantiated as objects of the `UltraTrailNet` class are compatible with the UltraTrail AI accelerator.

## 3.3 Designing and Constructing Search Spaces

## 3.4 Integrating HMM into the NAS Framework

## 3.5 Conclusion

# 4 Result Exploration and Analysis

# 5 Conclusion

# Bibliography


[1] Paul Palomero Bernardo et al. “UltraTrail: A Configurable Ultralow-Power TC-ResNet AI Accelerator for Efficient Keyword Spotting”. In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39.11 (2020), pp. 4240–4251.

[2] Paul Palomero Bernardo et al. “Compiler-aware AI Hardware Design for Edge Devices”. In: *EdgeSys ’25* (Mar. 2025), pp. 31–36.

[4] Jiaqi Zhang. “Survey of Quantization-Aware Training (QAT) Applications in Deep Learning Quantization”. In: *Proceedings of the 2025 International Symposium on Artificial Intelligence and Computational Social Sciences*. AICSS ’25. Association for Computing Machinery, 2025, pp. 431–442.

[5] Benoit Jacob et al. “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference”. In: June 2018, pp. 2704–2713.

[6] *Post Training Quantization (PTQ) — Torch-TensorRT — docs.pytorch.org*. https://docs.pytorch.org/{T}ensor{R}{T}/ts/ptq.html. [Accessed 10-06-2026].

[7] *Search Spaces - HANNAH — ekut-es.github.io*. https://ekut-es.github.io/hannah/neural-architecture-search/nas/search_spaces/. [Accessed 24-04-2026].

[8] Julia Werner et al. “Precise Localization Within the GI Tract by Combining Classification of CNNs and Time-Series Analysis of HMMs”. In: *Machine Learning in Medical Imaging*. Springer Nature Switzerland, Oct. 2023, pp. 174–183.

[9] *Numerically stable Viterbi algorithm in Python for hidden markov model state inference — crawlingrobotfortress.blogspot.com*. https://crawlingrobotfortress.blogspot.com/2016/07/python-recipe-for-numerically-stable.html. [Accessed 20-06-2026].

[10] *What is Batch Normalization In Deep Learning? - GeeksforGeeks — geeksforgeeks.org*. https://www.geeksforgeeks.org/deep-learning/what-is-batch-normalization-in-deep-learning/. [Accessed 07-06-2026]. 2025.

[11] *AdaptiveAvgPool1d — PyTorch 2.12 documentation — docs.pytorch.org*. https://docs.pytorch.org/docs/2.12/generated/torch.nn.{A}daptive{A}vg{P}ool1d.html#torch.nn.{A}daptive{A}vg{P}ool1d. [Accessed 07-06-2026].

[12] *AdaptiveAvgPool2d — PyTorch 2.12 documentation — docs.pytorch.org*. https://docs.pytorch.org/docs/2.12/generated/torch.nn.{A}daptive{A}vg{P}ool2d.html#torch.nn.{A}daptive{A}vg{P}ool2d. [Accessed 07-06-2026].

[13] *Custom Python Operators — PyTorch Tutorials 2.12.0+cu130 documentation — docs.pytorch.org*. https://docs.pytorch.org/tutorials/advanced/python_custom_ops.html#python-custom-ops-tutorial. [Accessed 18-06-2026].

[14] *torch.library — PyTorch 2.12 documentation — docs.pytorch.org*. https://docs.pytorch.org/docs/2.12/library.html#torch.library.custom_op. [Accessed 18-06-2026].

[15] Lei Mao. *PyTorch Custom Operation — leimao.github.io*. https://leimao.github.io/blog/{P}y{T}orch-{C}ustom-{O}peration/. [Accessed 18-06-2026].